

Git & GitHub Commands Reference

A complete visual cheat sheet for developers

14 Sections	130+ Commands	git + gh CLI Covered
-----------------------	-------------------------	--------------------------------

Crafted by Tejas Talole

Table of Contents

Setup & Configuration	0
	1
Repository Initialization	0
	2
Staging & Snapshots	0
	3
Committing	0
	4
Branching	0
	5
Merging & Rebasing	0
	6
Remote Repositories	0
	7
Inspection & Logging	0
	8
Stashing	0
	9
Tagging	1
	0
Undoing & Resetting	1
	1
GitHub CLI (gh)	1
	2
Advanced & Utilities	1
	3
.gitignore & Attributes	1
	4

Setup & Configuration

<code>\$ git config --global user.name "Your Name"</code>	Set your global Git username
<code>\$ git config --global user.email "you@example.com"</code>	Set your global Git email
<code>\$ git config --global core.editor "code"</code>	Set VS Code as the default editor
<code>\$ git config --list</code>	List all current Git configurations
<code>\$ git config --global alias.st status</code>	Create a shortcut alias (e.g. git st)
<code>\$ git config --global color.ui auto</code>	Enable colorized terminal output
<code>\$ git config --global core.autocrlf true</code>	Handle line-endings on Windows

Repository Initialization

<code>\$ git init</code>	Initialize a new local repository
<code>\$ git init</code>	Create a repo inside a specific folder
<code>\$ git clone</code>	Clone a remote repository locally
<code>\$ git clone</code>	Clone into a custom folder name
<code>\$ git clone --depth 1</code>	Shallow clone — only latest commit
<code>\$ git clone --branch</code>	Clone a specific branch

Staging & Snapshots

<code>\$ git status</code>	Show working-tree status (modified / staged)
<code>\$ git add</code>	Stage a specific file for commit
<code>\$ git add .</code>	Stage all changes in the current directory
<code>\$ git add -p</code>	Interactively stage chunks of changes
<code>\$ git add -A</code>	Stage all changes including deletions
<code>\$ git restore --staged</code>	Unstage a file without losing changes
<code>\$ git restore</code>	Discard working-directory changes
<code>\$ git rm</code>	Remove file from repo and disk
<code>\$ git rm --cached</code>	Untrack a file but keep it on disk
<code>\$ git mv</code>	Rename / move a file and stage the change

Committing

<code>\$ git commit -m "message"</code>	Commit staged changes with a message
<code>\$ git commit -am "message"</code>	Stage tracked files and commit in one step
<code>\$ git commit --amend</code>	Modify the most recent commit (msg or files)
<code>\$ git commit --amend --no-edit</code>	Amend last commit, keep the same message
<code>\$ git commit --allow-empty -m "msg"</code>	Create an empty commit (CI trigger)

Branching

<code>\$ git branch</code>	List all local branches
<code>\$ git branch -a</code>	List all branches — local + remote
<code>\$ git branch</code>	Create a new branch
<code>\$ git branch -d</code>	Delete a branch (safe — must be merged)
<code>\$ git branch -D</code>	Force-delete a branch
<code>\$ git branch -m</code>	Rename a branch
<code>\$ git switch</code>	Switch to an existing branch
<code>\$ git switch -c</code>	Create and switch to a new branch
<code>\$ git checkout</code>	Switch branch (legacy syntax)
<code>\$ git checkout -b</code>	Create and switch (legacy syntax)
<code>\$ git checkout -b origin/</code>	Track a remote branch locally

Merging & Rebasing

<code>\$ git merge</code>	Merge a branch into the current branch
<code>\$ git merge --no-ff</code>	Merge with a merge commit (no fast-forward)
<code>\$ git merge --squash</code>	Squash all branch commits into one
<code>\$ git merge --abort</code>	Abort an in-progress merge
<code>\$ git rebase</code>	Rebase current branch onto another
<code>\$ git rebase -i HEAD~</code>	Interactive rebase for last n commits
<code>\$ git rebase --abort</code>	Abort an in-progress rebase

\$ git rebase --continue	Continue rebase after resolving conflicts
\$ git cherry-pick	Apply a specific commit to current branch
\$ git cherry-pick ..	Cherry-pick a range of commits

Remote Repositories

\$ git remote -v	List remote connections with URLs
\$ git remote add origin	Add a remote named 'origin'
\$ git remote rename	Rename a remote
\$ git remote remove	Remove a remote
\$ git remote set-url origin	Change the URL of a remote
\$ git fetch origin	Download changes from remote (no merge)
\$ git fetch --all	Fetch from all remotes
\$ git pull	Fetch + merge from tracking branch
\$ git pull --rebase	Fetch + rebase instead of merge
\$ git push origin	Push a branch to remote
\$ git push -u origin	Push and set upstream tracking
\$ git push --force-with-lease	Force push safely — fails if remote changed
\$ git push origin --delete	Delete a remote branch
\$ git push --tags	Push all tags to remote

Inspection & Logging

\$ git log	Show commit history
\$ git log --oneline	Compact one-line commit history
\$ git log --oneline --graph --all	Visual branch graph
\$ git log -p	Show commits with diffs (patch)
\$ git log --author="name"	Filter commits by author
\$ git log --since="2 weeks ago"	Commits within a time range
\$ git log --grep="keyword"	Search commits by message keyword
\$ git log	History for a specific file

\$ git show	Show details of a specific commit
\$ git diff	Show unstaged changes
\$ git diff --staged	Show staged changes
\$ git diff ..	Diff between two branches
\$ git blame	Show who last changed each line
\$ git shortlog -sn	Summarize commits by author

Stashing

\$ git stash	Stash current uncommitted changes
\$ git stash save "message"	Stash with a descriptive label
\$ git stash list	List all stashes
\$ git stash pop	Apply latest stash and remove it
\$ git stash apply stash@{n}	Apply a specific stash (keep it)
\$ git stash drop stash@{n}	Delete a specific stash
\$ git stash clear	Remove all stashes
\$ git stash branch	Create a branch from a stash

Tagging

\$ git tag	List all tags
\$ git tag	Create a lightweight tag
\$ git tag -a -m "msg"	Create an annotated tag with message
\$ git tag -a	Tag a specific commit
\$ git show	Show tag details
\$ git push origin	Push a specific tag to remote
\$ git push origin --tags	Push all tags
\$ git tag -d	Delete a local tag
\$ git push origin --delete	Delete a remote tag

Undoing & Resetting

\$ git revert	Create a new commit that undoes a commit
\$ git revert HEAD	Revert the most recent commit
\$ git reset HEAD~1	Undo last commit, keep changes staged
\$ git reset --soft HEAD~1	Undo last commit, keep changes staged
\$ git reset --mixed HEAD~1	Undo last commit, unstage changes
\$ git reset --hard HEAD~1	Undo last commit and discard all changes
\$ git reset --hard origin/main	Reset local branch to match remote
\$ git clean -fd	Remove untracked files and directories
\$ git clean -n	Preview what clean would delete

GitHub CLI (gh)

\$ gh auth login	Authenticate with GitHub account
\$ gh repo create	Create a new GitHub repository
\$ gh repo clone /	Clone a GitHub repo
\$ gh repo view	View repository details in terminal
\$ gh repo fork	Fork a repository
\$ gh pr create	Open a new pull request
\$ gh pr list	List open pull requests
\$ gh pr checkout	Checkout a pull request locally
\$ gh pr merge	Merge a pull request
\$ gh pr review	Review a pull request
\$ gh issue create	Create a new issue
\$ gh issue list	List open issues
\$ gh issue close	Close an issue
\$ gh release create	Create a GitHub release
\$ gh workflow list	List GitHub Actions workflows
\$ gh run list	List recent workflow runs
\$ gh gist create	Create a GitHub Gist from a file

Advanced & Utilities

\$ git bisect start	Start binary search for a bug-introducing commit
\$ git bisect good	Mark a commit as good in bisect
\$ git bisect bad	Mark a commit as bad in bisect
\$ git bisect reset	End the bisect session
\$ git worktree add	Check out a branch in a new directory
\$ git worktree list	List all linked worktrees
\$ git submodule add	Add a submodule to your repository
\$ git submodule update --init	Initialize and update submodules
\$ git archive --format=zip HEAD > out.zip	Export repo as a zip archive
\$ git reflog	Show history of HEAD movements
\$ git gc	Garbage collect and optimize the repo
\$ git fsck	Verify integrity of the repository
\$ git ls-files	List all tracked files
\$ git count-objects -v	Count loose objects and disk usage

.gitignore & Attributes

\$ echo 'node_modules/' >> .gitignore	Ignore the node_modules folder
\$ git check-ignore -v	Find which rule is ignoring a file
\$ git rm -r --cached .	Un-track all files (re-apply .gitignore)
\$ git config --global core.excludesfile ~/.gitignore_global	Set global gitignore
\$ git config --global core.attributesfile ~/.gitattributes	Set global gitattributes

Pro Tips & Best Practices

Write Great Commit Messages	Use imperative mood: 'Fix bug' not 'Fixed bug'. Keep subject under 72 chars. Add a body explaining WHY, not just WHAT. Use ticket refs like [JIRA-123].
Use --force-with-lease	Always prefer git push --force-with-lease over --force. It fails if someone else pushed since your last fetch, protecting team history.
Pull Before You Push	Run git pull --rebase before pushing to avoid unnecessary merge commits and keep your history clean and linear.
Alias Your Frequent Commands	Set up aliases: git config --global alias.lg 'log --oneline --graph --all'. Try: alias.unstage, alias.last, alias.visual for faster workflows.
Use git bisect for Bug Hunting	When you don't know which commit introduced a bug, use git bisect start, mark good/bad commits, and Git will binary-search through history for you.
Keep Branches Short-Lived	Merge feature branches frequently. Long-lived branches cause painful merge conflicts. Use pull requests and code review to merge into main at least weekly.
Use .gitignore Proactively	Add .gitignore before your first commit. Use gitignore.io to generate rules for your OS, editor, and language automatically.
Tag Your Releases	Always tag release commits with git tag -a v1.0.0 -m 'Release v1.0.0'. Annotated tags include author, date, and message. Push them with --tags.

You're now a Git pro.

Keep this reference close and commit often.

docs.github.com • git-scm.com • cli.github.com

Crafted by Tejas Talole